AT

Anay Joshi
@anay.tech
AI tools, tech & career for students and early professionals

my handwritten study notes:

The Seven AI Engineering Interview Patterns

what's inside

1 Observability 4
2 Caching 7
3 Agentic evals 10
4 RAG 13
5 Fine-tuning 16
6 User feedback loops 19
7 Guardrails 22
★ Study plan and system-design template 25

say the answers out loud!

Notes by Anay Joshi - @anay.tech

Oct 2026 edition

How to use this guide

- The seven patterns are one request path plus the things that watch, test and improve it. Interviews rarely ask them in isolation: a system-design prompt like "design a support assistant" expects you to weave most of them together.
- The list of seven is a sound framing rather than an official standard. Recent interview write-ups describe the same core: ***evals are the round people flunk***, a judge-reliability question quickly becomes an observability conversation, and a defensible default design runs through an input layer, PII and safety checks, retrieval with a reranker, post-model guardrails and observability.
- ***How to use it:*** read a section's lead, watch one video, then say the interview answers out loud before reading the sample points.
- Videos are listed by title from their descriptions and are not verified, so skim each before committing time. Several are a year or two old, so verify prices, limits and model names against current provider docs.

The seven at a glance

#	Pattern	The question you will really	The answer to lead with
1	Observability	A user says an answer was wrong yesterday. How do you debug it?	Every request leaves a replayable trace: prompt version, retrieved context, tool calls, tokens, latency, cost.
2	Caching	How do you cut cost and latency without hurting quality?	Stack exact-match, semantic and prompt-prefix caches; tune the similarity threshold; scope by tenant.
3	Agentic evals	How do you know the agent got better?	A test set built from real failures, scoring outcome and trajectory, a judge calibrated to human labels, a CI gate.
4	RAG	The answers are wrong. Where do you look first?	Chunking, then hybrid retrieval and reranking, with retrieval measured separately from generation.
5	Fine-tuning	Should we fine-tune?	Not first. Prompt, then RAG for knowledge, then fine-tune only for behavior gaps.
6	User feedback loops	How does it improve after launch?	Signals tied to traces become eval rows; fixes ship only if the eval gate passes.

#	Pattern	The question you will really	The answer to lead with
7	Guardrails	How do you stop it saying something wild?	Layered input and output checks, least-privilege tools, and the assumption that the model can be tricked.

★ How the seven fit together

1. **Input guardrails** screen the request for injection attempts and PII.
2. **The cache** is checked: exact match, then semantic match.
3. **On a miss, RAG** retrieves with hybrid search and reranking.
4. **The model is called** with a stable prompt prefix so the provider's prompt cache can hit.
5. **Output guardrails** check the answer before the user sees it.
6. **The whole path is traced;** user feedback and evals read those traces, and failures feed back into steps 1 to 4. Fine-tuning is the last lever that loop pulls.

1 Observability

Key idea: Observability means every production request leaves a trace you can replay: the prompt, retrieved context, tool calls, tokens, latency, cost and any scores attached to it.

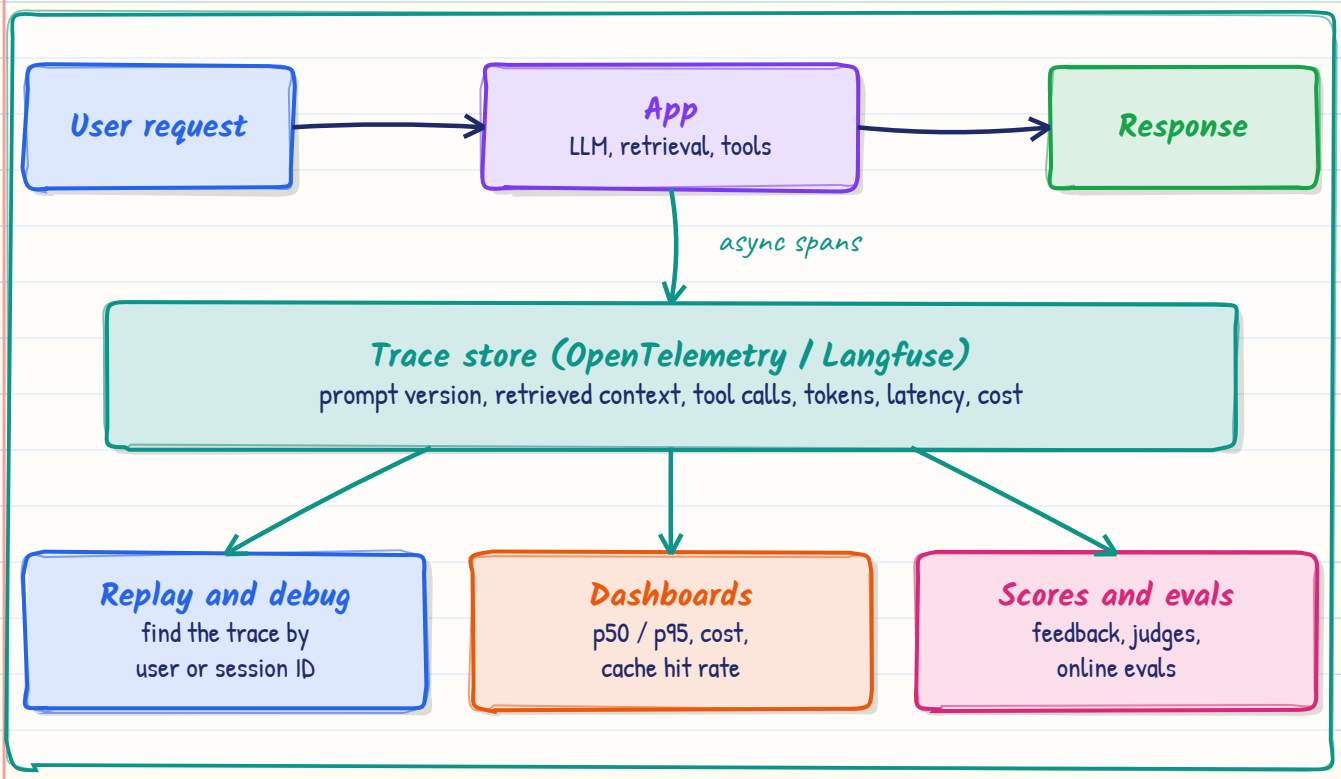


Figure 1. Every request emits spans asynchronously to a trace store; debugging, dashboards and evals all read the same traces.

How it works

- **Trace and spans.** A trace is one request end to end; its observations (LLM call, retrieval, tool call) are the steps inside it. Langfuse's tracing model groups traces into sessions and keeps the causal order between steps.
- **Tracing is one part of observability.** Observability also covers metrics and logs, but for LLM apps tracing is the most useful piece because it captures the full context of each request.
- **Build on OpenTelemetry.** Langfuse accepts OTLP traces, and libraries such as OpenLLMetry and OpenLIT add gen_ai.* instrumentation for many frameworks.
- **Zero request-path cost.** The Langfuse SDKs send tracing data asynchronously in the background, so tracing should not add latency to the user's request.

- **Scores live on traces.** User feedback, code checks and LLM-judge scores attach to a trace, which is what lets the same platform run evals on sampled production traffic (pattern 3) and feed datasets (pattern 6).
- **Measure what you will act on:** p50 and p95 latency, tokens and cost per request, error rate, cache hit rate, retrieval quality and eval scores, each sliceable by prompt version, model and user cohort.

★ Interview questions and strong points

Q: A user says yesterday's answer was wrong. What do you do?

→ Look up the trace by user or session ID, read the retrieved chunks and prompt version, then replay the request. The answer should be a procedure, not "check the logs".

Q: What do you log, and what don't you?

→ Redact PII before it leaves your boundary. For high-volume apps, sample, but never sample away errors and thumbs-down sessions. Tracing every call gets expensive at scale.

Q: Why not just use your existing APM?

→ General APM tools do not understand tokens, prompt versions or eval scores. Keep a general tracer for the non-LLM services and use an LLM-aware one for the model path.

Q: How do you catch a regression after a prompt change?

→ Version prompts, tag every trace with the version, compare score and cost distributions before and after.

× Common pitfalls

- × Traces without user or session IDs, so a complaint cannot be found.
- × Prompt text stored but no prompt version, so you cannot tell which change broke things.
- × Dashboards of averages that hide the tail.

Watch

- ***Langfuse Intro: Observability and Tracing Deep Dive*** - the product walkthrough from Langfuse's CEO; best first video.
- ***Langfuse Walkthrough: Agent Tracing, Evals, Experiments and Prompt Management*** - end-to-end demo for agents; shows how traces connect to evals.
- ***RAG Observability and Evaluations with Langfuse*** - tracing a RAG app and using evals to decide on chunking.

Read

- Langfuse: Observability and application tracing
- Langfuse: OpenTelemetry for LLM observability
- Langfuse tutorial: setup, SDK, trace capture

2 Caching

Key idea: LLM caching is three layers, and each saves different money: exact-match and semantic caches skip the model call entirely, while prompt caching only discounts the input side.

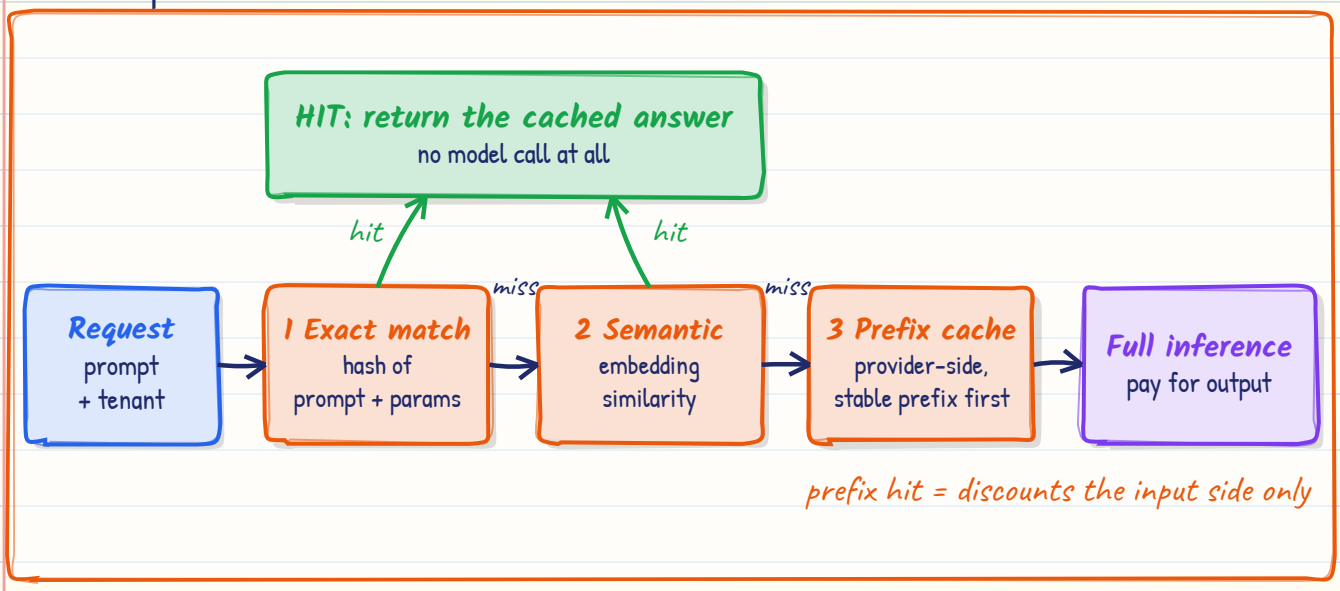


Figure 2. Caches are checked cheapest first; exact and semantic hits skip the model, while a prefix hit only discounts input tokens.

The three layers

Layer	What it matches	What it saves	Main risk
Exact-match	A hash of the normalized prompt plus model and parameters	The whole call, input and output tokens	Low hit rate; stale answers
Semantic	Embedding similarity to past queries	The whole call	False hits, cross-user leakage, cache poisoning
Prompt or prefix (provider-side)	A byte-identical prompt prefix	Input-side compute; you still pay for output	Any change early in the prompt breaks the hit

- The layers stack in this order: exact hash, then semantic match, then prefix cache, then full inference. Redis frames it the same way: exact match catches repeats, semantic catches paraphrases, prompt caching helps novel queries that share a long prefix.

How it works

- **Prompt caching depends on prefix order.** Put stable content first (system prompt, tool definitions, reference documents) and variable content last. Caching is exact-prefix, byte for byte, so a timestamp or user ID near the top silently kills hits.
- **Providers differ.** Per one gateway's docs, OpenAI caching is automatic above 1,024 tokens, Anthropic needs explicit cache_control markers, and Gemini's implicit caching starts at 2,048 tokens. Check current limits before quoting them.
- **Semantic caching has four steps:** embed the prompt, search stored vectors, return the cached answer on a hit, otherwise call the model and store the result. The similarity threshold is the most consequential setting and has no universal right value.
- **Scope matters.** Semantic caches are global by default, and an attacker who can influence what is cached can poison answers for other users. Key the cache by tenant and permission level.

Interview questions and strong points

Q: How do you choose the similarity threshold?

→ Build labelled pairs of true paraphrases and near-misses that need different answers, sweep the threshold, and pick the point with an acceptable false-hit rate. Start strict.

Q: What goes wrong with semantic caching?

→ The signature failure is the false hit: two prompts close in vector space that need different answers. Also staleness when source documents change, and leakage across users.

Q: When would you not use it?

→ When traffic is mostly novel or personalized: it then adds latency and embedding cost and returns nothing. It fits FAQ and support traffic where questions cluster.

Q: Your prompt-cache hit rate dropped. Why?

→ Something early in the prefix started varying (a timestamp, a request ID, reordered tools), or the prefix fell under the provider's minimum length.

Q: How do you invalidate?

→ TTLs plus explicit purges when documents, prompts or the model version change.

× Common pitfalls

- × Treating a vendor's accuracy figure for semantic matching as your own false-hit rate.
- × Caching responses that depend on user permissions.
- × Measuring only hit rate, not the quality of cached answers.

▶ Watch

- ▶ ***Prompt caching guide (non-technical)*** - how OpenAI, Anthropic and Google handle it; start here.
- ▶ ***Prompt Caching: A Complete Technical Guide*** - the mechanics, down to GPU-level behavior.
- ▶ ***Optimizing RAG with Semantic Caching and LLM Memory (Redis)*** - semantic caching in a RAG setting.
- ▶ ***Semantic Caching for LLM models*** - implementation-level walkthrough.
- ▶ ***Prompt Caching with Claude 3.5 Sonnet (tutorial)*** - older, tied to that model generation; use it for the cache_control idea, not current limits.

▮▮ Read

- ▮▮ Redis: What is prompt caching?
- ▮▮ Semantic caching for LLMs: the cost tier most teams skip
- ▮▮ How LLM caching works: prompt, KV and semantic caching

3 Agentic evals

Key idea: Prove an agent works by scoring the path it took as well as the answer it gave, on a test set built from real failures and gated in CI.

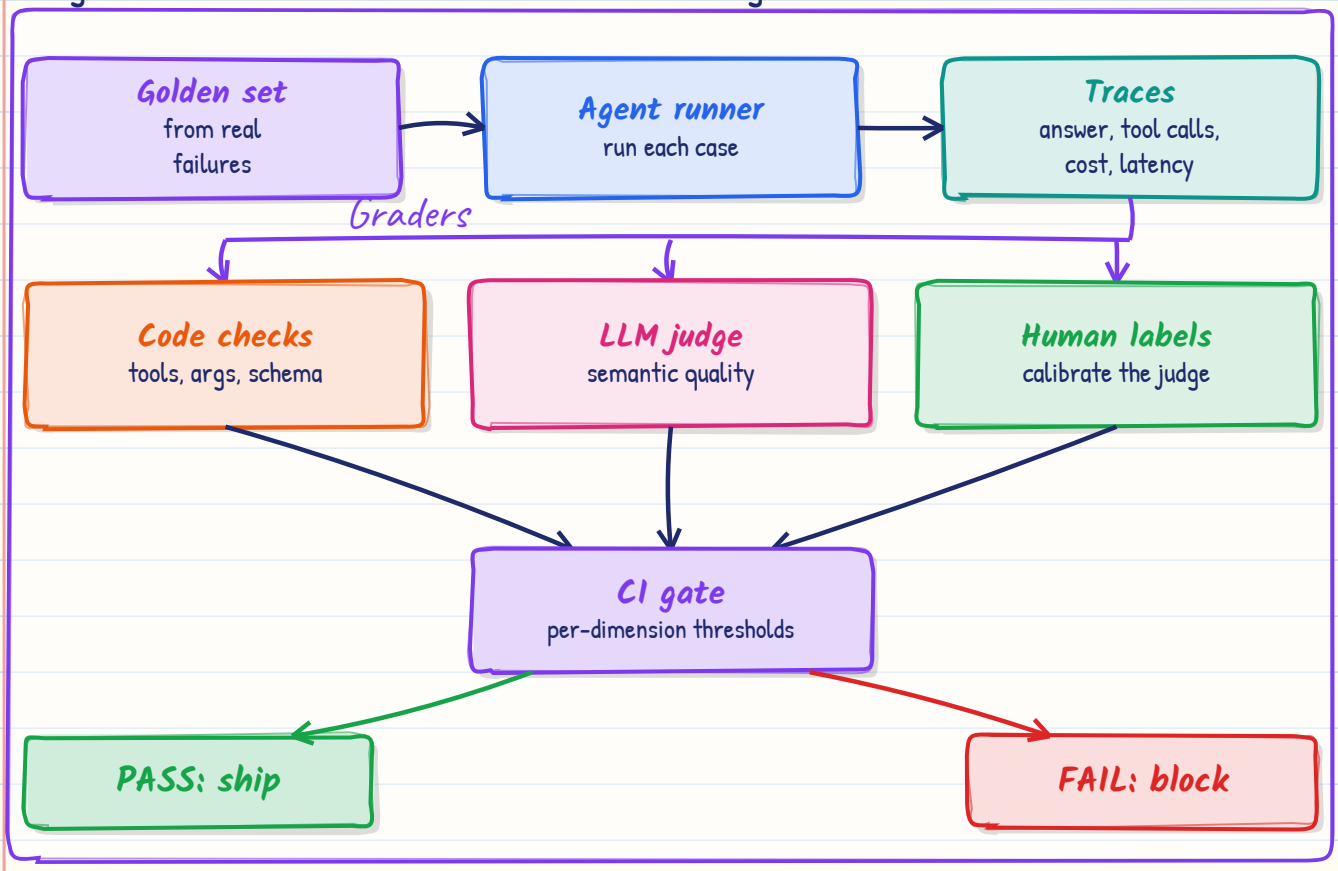


Figure 3. A golden set built from real failures runs through the agent; graders score outcome and trajectory, and a CI gate decides the release.

How it works

- **Outcome versus trajectory.** A pure outcome metric passes agents that got lucky; a pure trajectory metric penalizes valid alternative routes. Score both, plus cost.
- **Three kinds of grader.** Use deterministic code checks for anything decidable, an LLM judge for semantic judgments, and human annotation to calibrate both.
- **Trajectory matching.** LangChain's AgentEvals compares tool-call sequences against a reference in strict, unordered, subset or superset modes; superset accepts extra tool calls. It also offers a trajectory LLM judge that needs no reference.
- **The eval pipeline.** A golden dataset feeds a runner, the runner produces traces, a judge scores outcome, trajectory and cost, and a gate compares aggregates to

thresholds.

- ***Start from error analysis, not imagination.*** Hamel Husain and Shreya Shankar's method: read real traces, name the failure modes, then build evaluators for those. They prefer binary pass/fail over Likert scales and code checks before LLM judges.
- ***Calibrate the judge.*** An LLM judge can disagree with itself across runs and needs checking against human expert labels. Measure how often it catches true errors and how often it flags good outputs.
- ***Agent-as-a-Judge.*** A 2024 paper by researchers from Meta AI and KAUST (arXiv 2410.10934) had an agent with tools inspect another agent's work. On its own benchmark it reported 92.07% (gray-box) and 90.44% (black-box) alignment with human consensus, against 70.76% and 60.38% for a plain LLM judge. Treat these as the paper's reported numbers.
- ***Offline and online.*** One 2026 write-up puts it at roughly 52% of teams running offline evals but only 37% monitoring production with online evals. Strong answers cover both.

★ Interview questions and strong points

Q: How do you evaluate an agent?

→ Golden set from real failures; code checks for tool choice and arguments; trajectory score; LLM judge for the final answer's quality; CI gate with per-dimension thresholds.

Q: How do you know your LLM judge is any good?

→ Expect this follow-up every time. Label a sample by hand, compare the judge to your labels, report agreement, and re-check when the judge model or rubric changes.

Q: The agent passes evals but fails in production. Why?

→ The test set came from imagined failures, it is offline only, or the golden trajectory rejects valid alternative paths.

Q: How big should the eval set be?

→ Small and real beats large and hypothetical: early evals built on real failures have higher signal than broad coverage of imagined edge cases.

Q: What would you gate a release on?

→ Task success rate, tool-call accuracy, cost and latency per task, and zero regressions on the failure rows from past incidents.

× Common pitfalls

- × Generic metric dashboards (helpfulness, coherence) that never fail.
- × A golden trajectory so strict it fails correct but different routes.
- × Never refreshing the suite; passing evals graduate to a regression suite and new capability evals take their place.

▶ Watch

- ▶ ***Why AI evals are the hottest new skill*** (Hamel Husain and Shreya Shankar, Lenny's Podcast) - the full method, from error analysis to judges; the most useful video in this guide.
- ▶ ***Carrying Out Error Analysis*** - the live error-analysis demo that the podcast references.
- ▶ ***AI Agent Evaluation Crash Course*** - practical agent evaluation tutorial.
- ▶ ***How to evaluate agents in practice (Google ADK)*** - hands-on with a specific framework.
- ▶ ***Advanced Agent Evaluation Techniques: planning, reflection and trajectory evals*** - focused on trajectory scoring.

Read

- Hamel Husain: Your AI Product Needs Evals
- Husain and Shankar: AI Evals, everything you need to know
- Langfuse: AI agent evaluation
- LangChain: Agent evals with AgentEvals
- Arize: Agent-as-a-Judge

4 RAG

- **Key idea:** RAG is a pipeline, not a vector search: most production gains come from chunking, hybrid retrieval, reranking, and measuring each stage on its own.
- The stages, in order: ingestion, chunking, embeddings, indexing, retrieval, reranking, context construction, generation.

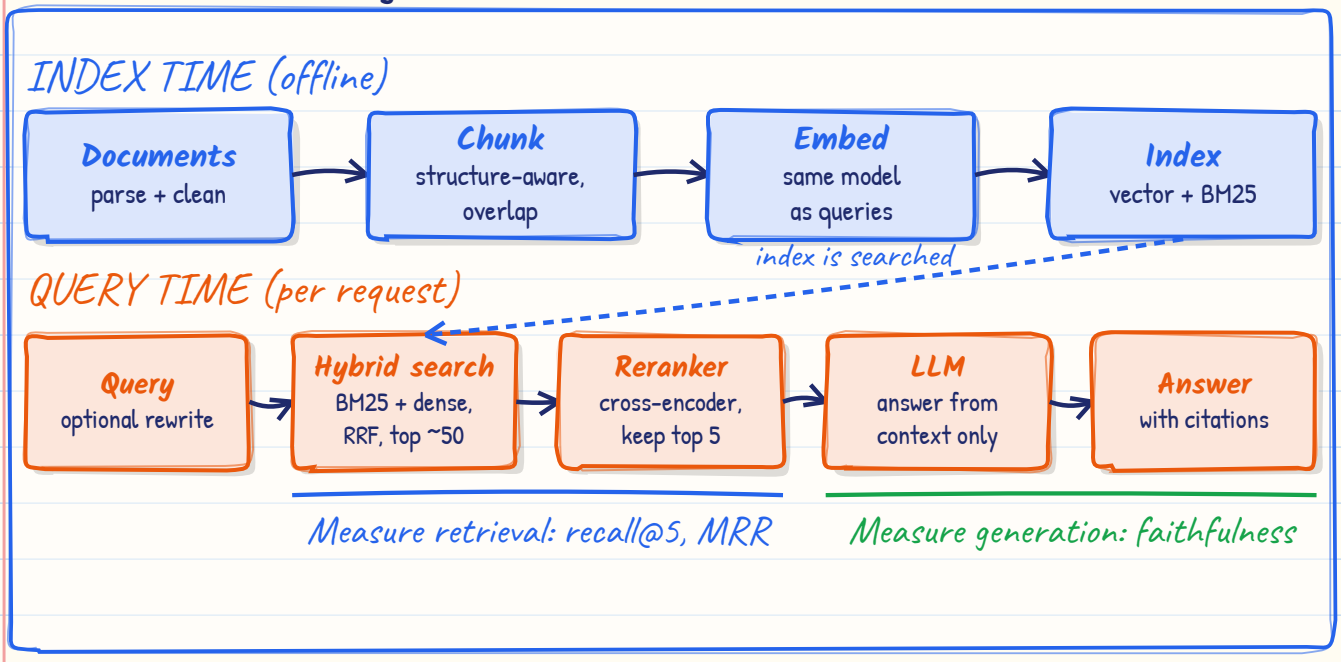


Figure 4. Index once, query many times; retrieval and generation are measured separately so you know where to look first.

How it works

- **Chunking is the biggest lever.** Fixed-size chunks with overlap are a starting point, not a strategy; chunk by semantic or structural boundaries and split only when a chunk is too big.
- **Dense search alone fails on exact strings.** Identifiers, error codes, product names and numbers are where pure vector search misses. Run BM25 and dense retrieval together and merge the results, commonly with reciprocal rank fusion (RRF).
- **Rerank a wide candidate set.** Retrieve around 50 with hybrid search, score them with a cross-encoder, keep the top 5. Retrieving 3 and reranking 3 is theater, and reranking 100 or more rarely pays off.
- **Rewrite queries only when it helps.** Expansion adds noise for short queries or ones containing exact identifiers; send those straight to hybrid search.

- ***Measure retrieval separately from generation.*** A typical take-home asks you to build a Q&A system over about 50 documents, write an eval set of 20 questions, and report recall@5 and faithfulness.
- ***Long context does not replace RAG.*** The 2026 consensus pattern is that RAG retrieves and long context refines.
- ***Budget the latency.*** One engineering write-up puts a full pipeline (query embedding, ANN search, optional rerank, generation) at 300 to 1,200 ms added at P50.
- ***Keep the index honest.*** Use the same embedding model for queries and documents, and re-embed when documents change.

★ Interview questions and strong points

Q: Design RAG for a company's internal knowledge base.

→ Cover ingestion and parsing, chunking by document type, hybrid retrieval, reranking, citations, access control on retrieved documents, and how you evaluate.

Q: Answers are wrong. Where do you look?

→ First ask whether the right chunk was retrieved. If not, the problem is chunking, embeddings or retrieval. If it was retrieved but the answer is still wrong, the problem is the prompt, context construction or the model. Say which metric would tell you each.

Q: Why hybrid search?

→ Dense retrieval handles meaning, BM25 handles exact terms; real queries mix both.

Q: How do you evaluate RAG?

→ Retrieval metrics (recall@k, MRR) on a labelled question set, plus generation metrics (faithfulness to the retrieved context, answer relevance), run offline and sampled online.

Q: How do you keep it fresh?

→ Incremental re-indexing on document change, with deletes handled, and a timestamp or version on every chunk.

× Common pitfalls

- × Different embedding models for indexing and querying.
- × Updating documents without refreshing their embeddings.

✗ Judging quality by eyeballing a few answers instead of a labelled set.

Watch

- ***RAG From Scratch playlist*** (Lance Martin, LangChain) - short videos covering indexing, retrieval, generation, query translation (multi-query, RAG-Fusion, HyDE) and self-correcting RAG.
- ***Learn RAG from Scratch: the 2.5-hour freeCodeCamp course*** - the same material as one long watch.
- ***RAG Observability and Evaluations with Langfuse*** - tracing a RAG app and evaluating chunking choices.
- ***Optimizing RAG with Semantic Caching and LLM Memory*** - caching layered onto RAG.

Read

- RAG in production in 2026: beyond naive chunk-and-embed
- RAG best practices in 2026: chunking, reranking, hybrid search
- The AI engineer interview guide I wish I had

5 Fine-tuning

★ **Key idea:** Fine-tune to change behavior, not to add knowledge: the ladder is prompting first, RAG second, fine-tuning last.

- Knowledge problems (the model doesn't know X) point to RAG; behavior problems (it won't reliably do X) point to better prompting or fine-tuning. The usual mistake is fine-tuning on documents or policies and expecting the model to recall them.

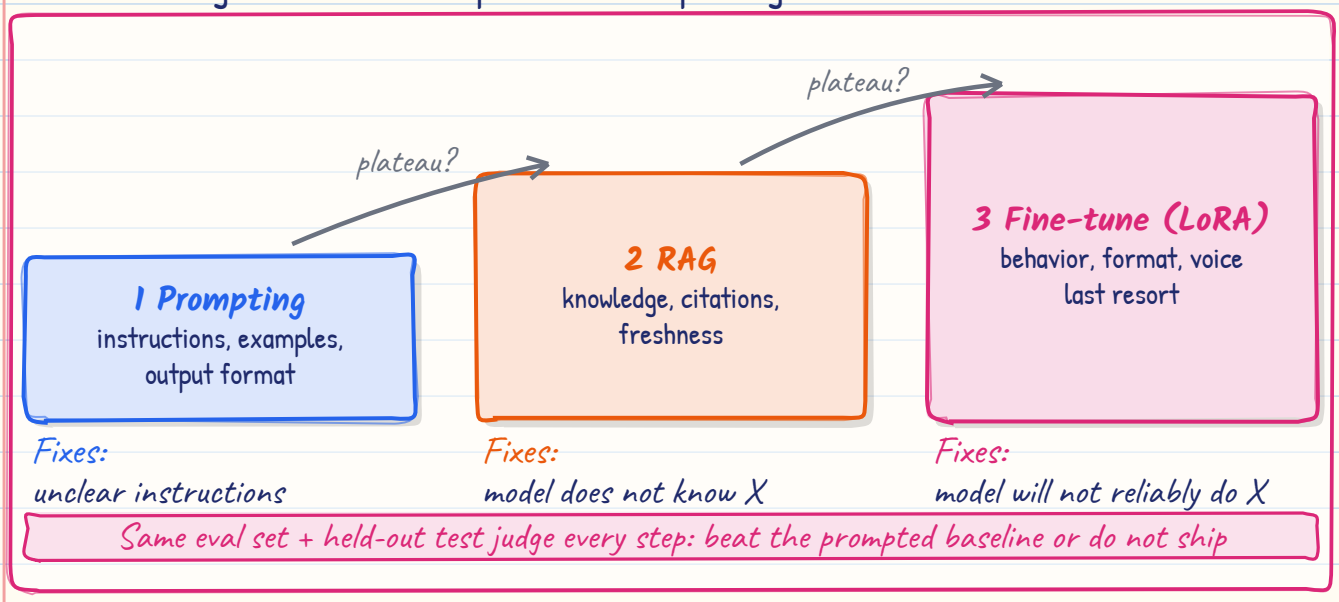


Figure 5. Climb the ladder in order, moving up only when the eval set shows the previous step has plateaued.

★ When to fine-tune, and when not to

Fine-tune when...	Do NOT fine-tune when...
You need rigid, machine-parseable output and prompting still drifts	You have not done a serious prompting pass
You need a consistent voice or a narrow skill prompts cannot lock in	The gap is missing or changing knowledge (use RAG)
A long prompt costs too much at your volume, or a small tuned model can replace a big one	The task definition changes every few weeks
Prompting and RAG have plateaued on your eval set after real iteration	You have no eval set or no way to retrain and maintain the model

★ How it works

- **Methods.** LoRA trains roughly 0.1% to 1% of the parameters a full run would, which is why it is the usual starting point. QLoRA adds a 4-bit quantized base so that 7B-plus models fit on a single GPU.
- **Data first.** Clean data and a working eval come before hyperparameters: tune LoRA rank and target modules only after you have both. Hold out a test set.
- **Hybrid is common.** Fine-tune for style and format, use RAG for facts; this is the standard shape for mature production systems.
- **A counterpoint.** Unsloth argues that claims that RAG always beats fine-tuning often come from badly configured fine-tunes (wrong LoRA settings, too little training). Know that both camps exist.
- **Where the data comes from.** The best training data is your own curated production feedback (pattern 6). Synthetic data from a teacher model is cheap but can make internal evals look better than real performance.

★ Interview questions and strong points

Q: We want the model to know our product docs. Fine-tune it?

→ No: use RAG. It is updatable without retraining and can cite sources. Fine-tuning facts into weights is expensive and stale on arrival.

Q: When would you fine-tune?

→ Name a behavior gap prompting could not close, show the eval that proves it, and say what you tried first.

Q: LoRA versus full fine-tuning?

→ LoRA for almost every team: far less memory, small quality gap on typical tasks, adapters are cheap to swap.

Q: How do you know it worked?

→ Compare to the prompted baseline on a held-out set that was never in training, and check you did not regress general ability.

Q: What does it cost to own?

→ Data curation, retraining whenever the base model or task changes, evaluation, and serving a custom model.

× Common pitfalls

- × Fine-tuning before building an eval, so nobody can say whether it helped.
- × Treating fine-tuning as a substitute for retrieval.
- × Forgetting the maintenance bill when the base model is upgraded.

▶ Watch

Video coverage I could verify is thin here, so lean on the reading.

- ▶ ***Prompt Engineering, RAG, and Fine-tuning: Benefits and When to Use*** - a clear overview of the three options; about three years old, so the concepts hold but tools have moved on.
- ▶ ***AI Engineering Explained: LLM, RAG, MCP, Agent, Fine-Tuning, Quantization*** - a live session that places fine-tuning in the wider stack.

Read

- Fine-tuning LLM: when to fine-tune vs RAG (2026)
- Together AI: RAG vs fine-tuning
- Fine-tuning vs RAG vs prompting: an engineering decision framework
- Unsloth: is fine-tuning right for me?
- Hamel Husain's blog: "Is Fine-Tuning Still Valuable?" (March 2024)

6 User feedback loops

Key idea: A feedback loop turns production traces, user signals and eval failures into better prompts, retrieval, training data and release gates; without the loop closing, you only have a pile of logs.

- The loop has five beats: ship, observe, diagnose, improve, ship again, and each turn adds new evals for the failure modes it uncovered. The practical mechanism is flagging a trace and turning it into a dataset record, running an experiment against it, and verifying the fix.

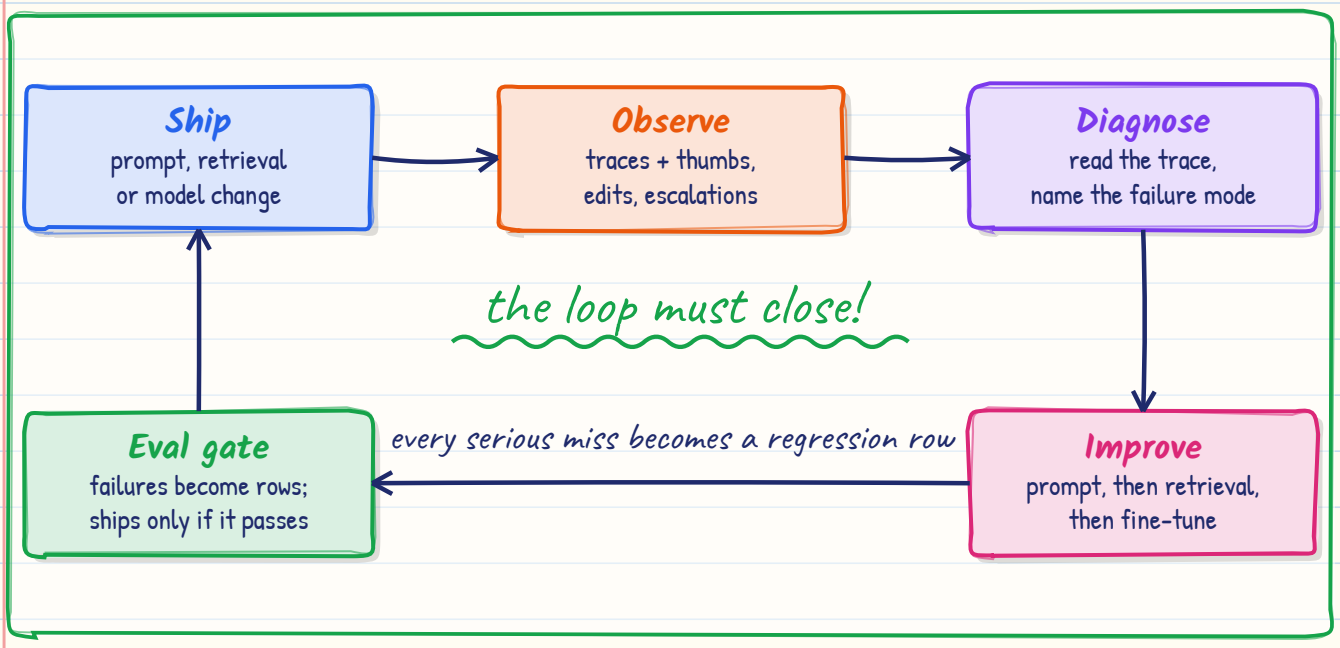


Figure 6. Signals tied to traces become eval rows; a fix ships only if the eval gate passes.

How it works

- ***Explicit feedback is rare.*** One practitioner write-up says fewer than 1% of production interactions yield explicit feedback, and elaborate forms see around 95% abandonment. Keep the thumbs widget simple.
- ***Use implicit signals.*** Regenerations, edits to the answer, abandonment, escalation to a human and reopened tickets cover far more traffic, with lower signal-to-noise. Thumbs-down rate, escalation rate and reopened-ticket rate are typical proxies.
- ***Tie every signal to a trace.*** Feedback is only useful if you can see the prompt, retrieved context and tool calls behind it (pattern 1).

- **Fix in order of cost.** Prompt and example changes first, then retrieval and knowledge-base fixes, then fine-tuning on curated production data (pattern 5).
- **Every serious miss becomes a regression row.** The fastest teams treat each production failure as a future eval case, so a fix cannot silently regress.
- **Keep a human in the loop where it counts.** An EMNLP 2025 industry paper on support agents collected four kinds of annotation inside live work (pairwise response preferences, whether the agent adopted the draft and why, knowledge relevance, missing knowledge). In a pilot it reported +11.7% recall@75, +14.8% precision@8, +8.4% helpfulness and +4.5% adoption, and cut retraining cycles from months to weeks.
- **A second example.** NVIDIA's internal assistant for 30,000 employees collected 495 negative samples over three months and reported a 3.7% accuracy gain and 40% latency reduction from fine-tuning its query rephrasal step.

★ Interview questions and strong points

Q: How does your system improve after launch?

→ Describe the loop concretely: capture, triage, promote to dataset, experiment, gate, deploy, with a named owner and a cycle time.

Q: Users almost never click thumbs. Now what?

→ Mine implicit signals, sample traces for expert review, and use an LLM judge on production traffic, calibrated against human labels.

Q: How do you avoid training on bad feedback?

→ Filter by trace quality, check labels against expert review, and watch for sycophancy creeping in through the loop when users upvote agreeable answers.

Q: How do you know a fix did not break something else?

→ The regression suite built from past failures runs in CI before every deploy.

Q: What do you measure about the loop itself?

→ Cycle time from failure to shipped fix, data quality, and regressions prevented; not raw feedback volume.

× Common pitfalls

- × Counting feedback volume as progress.
- × Conflating dimensions of feedback (helpful, correct, safe, fast) into one score.

✕ Letting the loop run fully automatic with no review of what gets promoted into training data.

Watch

I could not find a video dedicated to feedback loops that I could verify. These cover the same loop from the eval side:

- ***Why AI evals are the hottest new skill*** (Hamel Husain and Shreya Shankar) - the flywheel idea runs through the error-analysis and monitoring discussion.
- ***Langfuse Walkthrough: Agent Tracing, Evals, Experiments and Prompt Management*** - shows traces moving into datasets and experiments.

Read

- Data flywheels for LLM applications (Tian Pan)
- Agentic AI flywheels (SwirlAI)
- Building the data flywheel (Agenta)
- Agent-in-the-Loop: a data flywheel for LLM customer support (arXiv)
- Adaptive Data Flywheel: MAPE control loops for agents (arXiv)

7 Guardrails

Key idea: Guardrails are layered checks around the model (on what goes in, what comes out, and what the model is allowed to do), because no single filter stops a determined attacker or a bad day.

- Effective prompt-injection defense is layered: input validation filters malicious patterns before the model, output filtering prevents data leakage, execution sandboxing contains attacks that succeed, and continuous monitoring catches evolving ones. The basic request flow is user, input guards, LLM, output guards, user. Input guards block synchronously; output guards can run asynchronously when you stream.

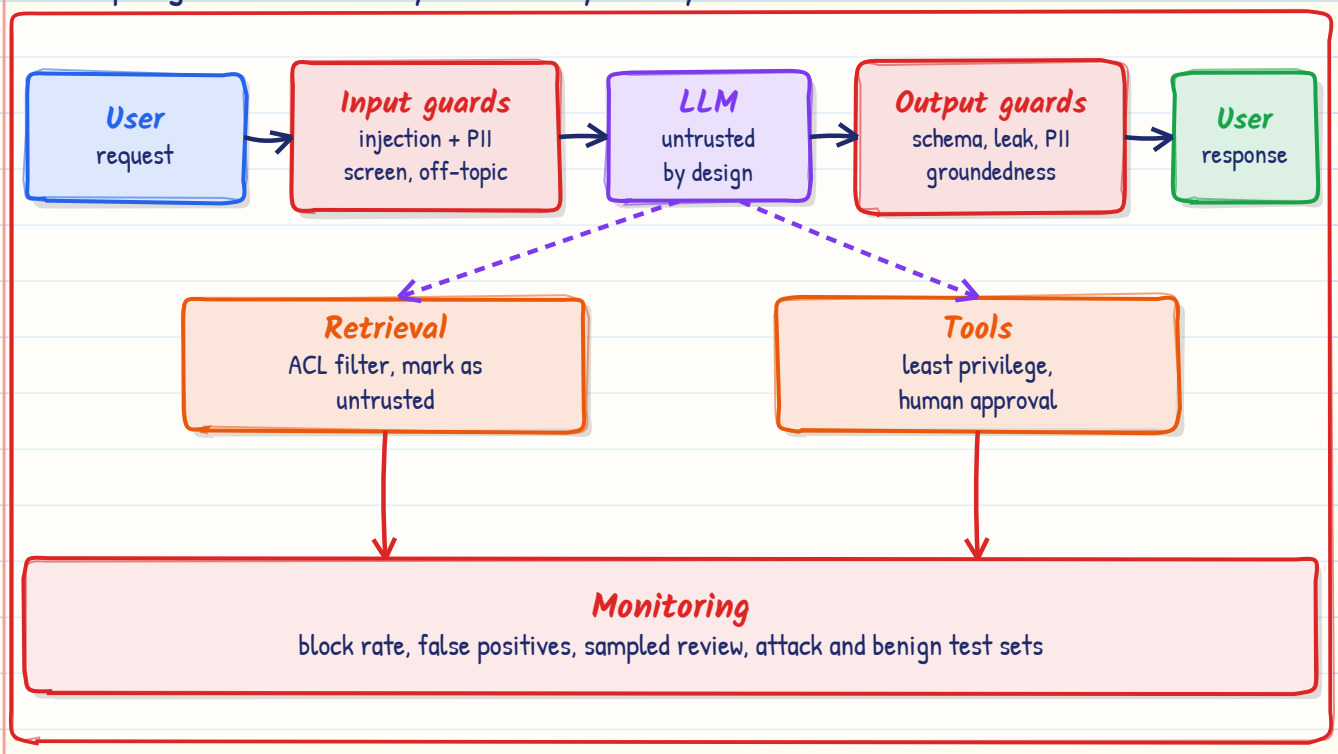


Figure 7. Layered checks on input, output and actions, with monitoring over the whole path; assume the model can be tricked.

How it works

- **Input side.** Screen for injection and jailbreak attempts, strip or mask PII before it reaches the model, and reject off-topic requests.
- **Output side.** Validate against a schema, check for leaked system prompt text or PII, and check groundedness against retrieved context for RAG answers.
- **Action side.** The strongest control is limiting what a tricked model can do: bounded actions, least-privilege tool permissions, escalation paths and human review for risky

steps.

- ***Indirect injection.*** Malicious instructions can hide in retrieved documents or tool results, not only in the user's message. Marking untrusted content (spotlighting) and sandboxing tool output are standard mitigations.
- ***Tooling.*** NeMo Guardrails (NVIDIA, Colang policies, input, output, dialog, retrieval and execution rails), Guardrails AI (schema and validator based), and small classifier models such as Meta's Prompt Guard 2 (about 86M parameters per one security paper). Tools differ mainly in whether they target conversation flow, structured output, or input and output scanning.
- ***Cost is real.*** In NeMo's common self-check setup, every screened message costs an extra model call. Budget the latency, and put cheap classifiers in front of expensive judges.
- ***Prompt attacks rank as the leading security threat*** for LLM applications, per the OWASP 2025 list that paper cites.

★ Interview questions and strong points

Q: Design guardrails for a RAG support bot.

→ Input: injection screening and PII masking. Retrieval: access control on documents. Output: groundedness check, schema validation, refusal for out-of-scope topics. Actions: least-privilege tools. Monitoring: block rates, false positives and sampled review.

Q: How do you handle prompt injection through retrieved documents?

→ Treat retrieved text as untrusted data, mark it, never let it grant tool permissions, and test with poisoned documents in your eval set.

Q: What is your latency budget for guardrails?

→ Run cheap deterministic checks first, classifiers second, LLM judges only for borderline cases, and stream output with asynchronous checks where safe.

Q: How do you know the guardrail works?

→ Maintain an attack test set and a benign test set; report catch rate and false-positive rate. Over-blocking has a cost, and one study of an educational tutor built its whole evaluation around that trade-off.

Q: Can guardrails be bypassed?

→ Yes. A pre-trained classifier only detects what it was trained to detect, and attacks like best-of-N rewording defeat filters, so defense in depth and limited permissions

matter more than any one filter.

✕ Common pitfalls

- ✕ Relying on the system prompt alone.
- ✕ Guardrails with no monitoring, so nobody sees the false positives or the misses.
- ✕ Giving the agent broad tool permissions and hoping the filter catches abuse.

▶ Watch

- ▶ ***AI Agent Guardrails Simplified: Prompt Injection, PII and more*** - the three-layer framing, good first video.
- ▶ ***How to Implement Guardrails in a RAG System*** (AI Engineering Interview Series) - framed as an interview answer.
- ▶ ***Bedrock Guardrails Demo: secure your AI apps and agents*** - a managed-service walkthrough of injection defense.
- ▶ ***Spikee #4: Bypassing LLM Guardrails*** - how system messages, spotlighting and injection filters fare against attacks; useful for the "can it be bypassed" question.

📖 Read

- 📖 LLM guardrails: preventing prompt injection and unsafe outputs
- 📖 Implementing guardrails against prompt injection
- 📖 NVIDIA NeMo Guardrails tutorials
- 📖 NeMo Guardrails tutorial (StationX)

★ Study plan and mock-interview drills

- Two weeks is enough to become fluent if every pattern gets one small build and one spoken answer. The plan below is suggested pacing, not something drawn from a source.

Day	Pattern	Build	Spoken answer to recor
1	Observabilit y	Trace a small RAG or agent app in Langfuse with user and session IDs	"Debug yesterday's wrong answer" in 90 seconds
2	Caching	Add an exact-match cache and prompt-prefix ordering; log hit rate and cost	"Three cache layers and their risks"
3	Agentic evals I	Read 30 real traces; write down failure modes	"How I build an eval set from failures"
4	Agentic evals II	20-row golden set, one code check, one LLM judge checked against your own labels	"How I know my judge is good"
5-6	RAG	Baseline vector search, then hybrid plus reranking; report recall@5 for both	"Answers are wrong, where do I look?"
7	Fine-tuning	Write a one-page decision memo for a real use case; optional toy LoRA run	"Should we fine-tune?"
8	Feedback loops	Add a thumbs widget and one implicit signal; promote three failed traces into the eval set	"How the system improves after launch"
9	Guardrails	Input and output checks plus 20 attack and 20 benign test prompts	"Design guardrails for a RAG support bot"
10	Integration	Whiteboard "design a support assistant" using the template below, timed at 35 minutes	The full design, out loud
11-14	Mocks	Two mock interviews with a friend or the videos below; revisit your weakest pattern	Redo day 10 with a different prompt

★ System-design answer template

- Use this order for any "design an AI feature" prompt. It follows the shape of the seven-step approach in the KDnuggets guide but names the seven patterns explicitly.

1. **Clarify** the users, the data, the success metric and the failure that would hurt most.
2. **Draw the request path:** input checks, cache, retrieval, model, output checks.
3. **Retrieval (RAG):** chunking, hybrid search, reranking, access control.

4. ***Cost and latency (caching, routing):*** cache layers, a stable prompt prefix, a smaller model for easy requests.
5. ***Safety (guardrails):*** input and output checks, least-privilege tools.
6. ***Quality (evals):*** offline golden set from real failures, calibrated judge, CI gate; trajectory scoring if it is an agent.
7. ***Operations (observability):*** traces with versions and IDs, dashboards, online evals on sampled traffic.
8. ***Improvement (feedback loop, then fine-tuning):*** capture signals, promote failures to the eval set, and only then consider tuning for behavior gaps.
9. ***Failure modes:*** name two things that could go wrong and how you would detect each.

☆ ***Practitioner tip:*** you do not need to pretend every project had online evals and semantic caching. Say what you built, then say what you would add if the system had to scale or become more critical. A real failure you diagnosed is one of the strongest things to discuss in a senior interview.

▶ Interview-prep videos

- ***AI / LLM Engineer Mock Interview: easy to hard*** - embeddings, RAG, agents and evaluation, with attention to how a strong candidate treats "is it actually better?"
- ***GenAI Interview Guide for Senior AI and LLM Engineer roles*** - the most-asked question list.
- ***AI System Design with GenAI and RAG Masterclass*** - a system-design walkthrough.

Further reading on the interview itself

- What applied-AI engineer interviews test in 2026
- How to answer AI system design interview questions (KDnuggets)
- Interview questions for an AI engineer (Pitchnhire)

Now go build one thing per pattern!

Notes by Anay Joshi - @anay.tech